Design a URL Shortener

A Simple and Easy HLD Interview Guide

> This guide explains how to design a URL shortener like TinyURL or Bitly in simple language. It also tells you exactly how to explain the design step by step in an HLD interview.

1. What is the Problem?

Suppose we have a very long URL:

```
https://www.example.com/products/category/mobile/samsung/...
```

We want to create a short URL such as:

```
https://short.ly/abc123
```

When a user opens the short URL, the system should automatically redirect the user to the original long URL.

Two Main Operations

• **URL Shortening:** Long URL → Short URL

• **URL Redirection:** Short URL → Original Long URL

2. How Should You Start in an HLD Interview?

Do not immediately start drawing databases and servers. Start by understanding the problem.

You can say: "First, I would like to clarify the functional and non-functional requirements."

Functional Requirements

• The user should be able to create a short URL from a long URL.

• When the user opens a short URL, the system should redirect to the original URL.

• The generated short URL should be unique.

• The short code should be as short as possible.

• We should clarify whether URLs can expire, be updated, deleted, or customized.

Non-Functional Requirements

• High availability

• Low latency

• Scalability

• Reliability

• Ability to handle a large number of requests

> **Simple interview assumption:** "For the initial design, I will assume URLs are immutable and do not expire unless the interviewer asks for expiration."

3. Back-of-the-Envelope Estimation

The book assumes approximately 100 million new URLs are generated every day.

Write QPS

```
100 million / (24 × 60 × 60)
≈ 1,160 writes per second
```

Therefore, write QPS is approximately **1,160 requests per second**.

Read QPS

A URL shortener usually has many more reads than writes. Suppose:

```
Read : Write = 10 : 1
```

```
Read QPS = 1,160 × 10
≈ 11,600 reads per second
```

Important conclusion: This is a read-heavy system. Therefore, caching will be very useful.

Storage Estimation

If we generate 100 million URLs every day for 10 years:

```
100 million × 365 × 10
≈ 365 billion records
```

If one record takes roughly 100 bytes, we may need tens of terabytes of storage.

> **Interview conclusion:** "The system may store hundreds of billions of records, so the database must be horizontally scalable."

4. API Design

Create a Short URL

```
POST /shorten

Request:
{
  "longURL": "https://www.google.com/very-long-url"
}

Response:
{
  "shortURL": "https://short.ly/abc123"
}
```

Redirect a Short URL

```
GET /abc123
```

The system finds the original URL and sends an HTTP redirect response.

5. Database Design

A simple table can look like this:

Field	Description

id	Unique internal ID
short_code	The short code, for example abc123
long_url	Original long URL
created_at	When the URL was created
expires_at	Optional expiration time

The most important lookup is:

```
short_code → long_url
```

Therefore, **short_code should be indexed and unique**.

6. URL Redirection

Suppose a user opens:

```
https://short.ly/abc123
```

The flow is:

```
User
  ↓
Browser sends GET /abc123
  ↓
URL Shortener Server
  ↓
Find abc123
  ↓
Get Original URL
  ↓
Send HTTP Redirect
  ↓
Browser opens Original URL
```

7. 301 vs 302 Redirect

301 - Permanent Redirect

301 means the resource has permanently moved. Browsers may cache the redirect.

• Can reduce requests to the URL shortener.

• Useful when the mapping never changes.

302 - Temporary Redirect

302 tells the browser to redirect temporarily.

• The client usually continues to contact our URL shortener.

• Useful when we want to track every click and collect analytics.

> **Good interview answer:** "If URLs are immutable and reducing server load is important, 301 can be used. If we need click analytics, 302 is often preferable because requests continue coming to our service."

8. How Do We Generate the Short URL?

This is one of the most important parts of the design. There are two common approaches:

Approach 1: Hashing

```
Long URL
   ↓
Hash Function
   ↓
Hash Value
   ↓
Short Code
```

We can hash the long URL using a hash function. However, hash values can be very long, so we may use only part of the hash.

The Problem: Hash Collision

Two different URLs might produce the same short code.

```
URL 1 → abc123
URL 2 → abc123
```

This is called a **collision**.

How to Handle Collision

```
Generate short code
        ↓
Check Database
        ↓
Already exists?
   Yes          No
    ↓            ↓
Generate        Save
another code
```

9. Base62 Encoding

Base62 uses the following characters:

```
0-9   → 10 characters
a-z   → 26 characters
A-Z   → 26 characters

Total = 62 characters
```

Because we have 62 possible characters, we can represent large numbers using fewer characters.

For example, a 7-character code has:

```
62^7 ≈ 3.5 trillion combinations
```

This provides a very large number of possible short URLs.

10. Recommended Approach: Unique ID + Base62

For an interview, a very simple approach is:

```
Generate Unique ID
        ↓
Convert ID to Base62
        ↓
```

```
Generate Short Code
```

For example:

```
Unique ID = 1000000
       ↓
Base62 Encoding
       ↓
Short Code = 4C92   (example)
```

Then the final URL becomes:

```
https://short.ly/4C92
```

> **Why is this approach good?** If the ID generator guarantees uniqueness, the generated short codes will also be unique after Base62 encoding.

11. Complete URL Creation Flow

Suppose the user sends a long URL.

```
Client
   |
   | POST /shorten
   ↓
Load Balancer
   ↓
Application Server
   ↓
Generate Unique ID
   ↓
Convert ID to Base62
   ↓
Store Mapping in Database
   ↓
Return Short URL
```

Step-by-Step

• The client sends POST /shorten with the long URL.

• The request reaches the Load Balancer.

• The Load Balancer sends the request to one of the application servers.

• The application generates a unique ID.

• The ID is converted to Base62.

• The short code and long URL are stored in the database.

• The system returns the final short URL to the user.

12. Complete Redirect Flow

Now suppose the user opens:

```
https://short.ly/aBc12
```

```
Client
   |
   | GET /aBc12
   ↓
```

```
Load Balancer
   ↓
Application Server
   ↓
Check Redis Cache
   |
   ■■■ Cache Hit → Get Long URL → Redirect User
   |
   ■■■ Cache Miss
          ↓
       Database
          ↓
       Get Long URL
          ↓
       Put Result in Redis
          ↓
       Redirect User
```

13. Why Do We Use Redis?

We estimated that the system has approximately:

```
Writes: 1,160/sec
Reads:  11,600/sec
```

Therefore, the system receives many more reads than writes.

If every redirect request goes directly to the database, the database may receive too much traffic.

Redis stores frequently accessed URLs in memory.

```
Redis

abc123 → https://google.com
xyz789 → https://amazon.com
```

Memory access is much faster than database access, so Redis can significantly reduce latency and database load.

14. Cache Hit and Cache Miss

Cache Hit

The short code already exists in Redis.

```
GET /abc123
    ↓
Redis
    ↓
Long URL Found
    ↓
Redirect User
```

Cache Miss

The short code is not available in Redis.

```
GET /abc123
    ↓
Redis
    ↓
```

```
Not Found
    ↓
Database
    ↓
Get Long URL
    ↓
Save in Redis
    ↓
Redirect User
```

15. How Do We Generate Unique IDs in Multiple Servers?

Suppose we have multiple application servers. We must make sure that two servers do not generate the same ID.

Option 1: Database Auto Increment

```
Database generates:
1, 2, 3, 4, 5 ...
```

Simple, but the database can become a bottleneck at very large scale.

Option 2: Dedicated Distributed ID Generator

```
Application Servers
        ↓
   ID Generator
        ↓
     Unique ID
```

A dedicated service generates globally unique IDs.

Option 3: Range Allocation

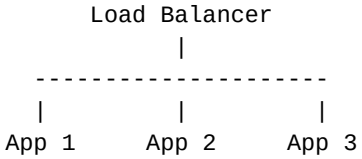
Different servers receive different ranges.

```
Server 1 → 1 to 1,000,000

Server 2 → 1,000,001 to 2,000,000

Server 3 → 2,000,001 to 3,000,000
```

Each server can generate IDs independently within its allocated range.

16. Scaling the Application

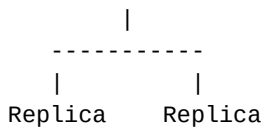
We should not have only one application server because it creates a single point of failure.



The application servers should ideally be **stateless**. This means any request can go to any server.

17. Database Replication

Since this system is read-heavy, database replicas can help.



Write Operation

```
Application → Primary Database
```

Read Operation

```
Application → Read Replica
```

However, in practice, Redis should handle many frequently accessed reads before they reach the database.

> **Good interview statement:** "Initially, I would use database replication because the system is read-heavy."

18. Database Sharding

Over time, the system may contain hundreds of billions of URL records. One database may not be enough.

Sharding means splitting the data across multiple databases.

Example

```
Shard 1
Shard 2
Shard 3
Shard 4
```

We can use a hash of the short code to decide which shard stores the data:

```
hash(short_code) % number_of_shards
```

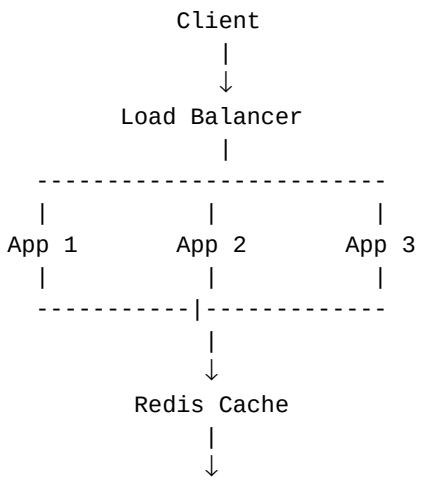
For example:

```
hash(abc123) % 4 = 2
```

Therefore, the record is stored in Shard 2.

> **Good interview statement:** "As the data grows, I would introduce sharding to distribute the dataset across multiple database nodes."

19. Final High-Level Architecture



```
        Database Layer
              |
         Primary DB
              |
        -------------
        |           |
    Replica      Replica
```

20. How to Explain the Complete Architecture

You can say the following in an interview:

> **"The client sends the request through a load balancer, which distributes traffic across multiple stateless application servers."**
>
> **"For URL creation, the application generates a globally unique ID and converts it to Base62 to create a short code."**
>
> **"The mapping between the short code and the original URL is stored in the database."**
>
> **"For URL redirection, because the system is read-heavy, the application first checks Redis."**
>
> **"On a cache hit, the original URL is retrieved immediately. On a cache miss, the application fetches the URL from the database and then stores it in the cache."**
>
> **"For scaling, I would use multiple application servers, database replication for reads, and eventually sharding as the dataset grows."**

21. Complete Example

Create URL

User sends:

```
POST /shorten

{
  "longURL": "https://www.amazon.in/products/mobile/samsung/xyz"
}
```

The system generates:

```
Unique ID = 123456789
        ↓
Base62
        ↓
8M0kX
```

Database stores:

```
short_code = 8M0kX

long_url =
https://www.amazon.in/products/mobile/samsung/xyz
```

The system returns:

```
https://short.ly/8M0kX
```

Redirect URL

The user opens:

```
https://short.ly/8M0kX
```

The system performs:

```
GET /8M0kX
     ↓
Redis
     ↓
Find Long URL
     ↓
HTTP 302 Redirect
     ↓
Browser opens Amazon URL
```

22. The Exact HLD Interview Flow

Whenever an interviewer says, **'Design a URL Shortener'**, follow this order:

1. Clarify Requirements: Ask about expiration, custom URLs, updates, deletion, analytics, and traffic.

2. Functional Requirements: Create a short URL and redirect a short URL.

3. Non-Functional Requirements: Scalability, availability, low latency, and reliability.

4. Capacity Estimation: Calculate write QPS, read QPS, and storage.

5. API Design: Define POST /shorten and GET /{shortCode}.

6. Database Design: Define short_code, long_url, and indexes.

7. Short Code Generation: Discuss hashing and Unique ID + Base62. Prefer Unique ID + Base62 for simplicity.

8. Write Flow: Explain how a long URL becomes a short URL.

9. Read Flow: Explain how the system redirects the user.

10. Caching: Use Redis because the system is read-heavy.

11. Scaling: Use Load Balancer, multiple app servers, replication, and sharding.

12. Trade-offs: Discuss 301 vs 302 and hashing vs Base62.

23. What Should You NOT Do Initially?

Do not immediately start with a very complicated architecture.

Avoid starting with:

• Kafka

• Kubernetes

• Many microservices

• CDN

• Complex sharding

• Too many components

Start simple:

```
Client
  ↓
Load Balancer
  ↓
Application Server
  ↓
Database
```

Then improve the design:

```
Step 1 → Add multiple application servers
Step 2 → Add Redis
Step 3 → Add database replicas
Step 4 → Add sharding when required
```

> **This shows good engineering thinking because you are scaling the system based on actual requirements.**

24. Important Interview Question: What About Hash Collisions?

Do not simply say:

■ "I will hash the URL and use the first 7 characters."

The interviewer may ask:

"What if two URLs generate the same short code?"

Your answer should be:

> **"Hash collisions are possible. We need to check whether the generated short code already exists and handle collisions. Alternatively, I prefer generating a globally unique numeric ID and encoding it with Base62. If the ID generator guarantees uniqueness, the short code will also be unique."**

25. 10-Minute Interview Answer Strategy

Time	What to Explain
Minute 1	Clarify requirements
Minute 2	Back-of-the-envelope estimation
Minute 3	API and database design
Minute 4-5	Short code generation
Minute 6	Write flow
Minute 7	Read flow
Minute 8	Caching with Redis
Minute 9	Replication and scaling
Minute 10	Trade-offs and interviewer questions

26. Short Final Answer You Can Speak

If you want to summarize the complete design, you can say:

"I will first clarify the requirements. The system needs to create a short URL and redirect users to the original URL. Based on the expected traffic, this is a read-heavy system. For the API, I will have one endpoint to create a short URL and another endpoint to redirect users. I will generate a globally unique ID and convert it to Base62 to generate a short code. The mapping between the short code and long URL will be stored in the database. For redirection, I will first check Redis. On a cache miss, I will read from the database and populate Redis. To scale the system, I will use a load balancer, multiple stateless application servers, database replicas, and eventually sharding as the data grows."

27. Final Revision Checklist

• What are the functional requirements?

• Why is this system read-heavy?

• How do we calculate QPS?

• What APIs do we need?

• What does the database schema look like?

• What is Base62?

• Why use Base62?

• How do we generate unique IDs?

• What is a hash collision?

• How do we handle collisions?

• Why use Redis?

• What is cache hit and cache miss?

• What is the difference between 301 and 302?

• Why use a Load Balancer?

• Why should app servers be stateless?

• What is database replication?

• What is database sharding?

If you understand these points clearly, you can confidently explain a URL Shortener design in an HLD interview.